

Algorithmic Trading System

System Overview

This project aims to build a comprehensive, enterprise-grade **algorithmic trading system** from the ground up. The platform's core philosophy is a "Best-of-Breed" integration strategy, selecting the best open-source projects for each major component to create a powerful, modular foundation. The system caters to both non-technical users (via simplicity and no-code options) and professional traders/analysts (via enterprise-grade features), leveraging AI Agents and Agentic AI extensively.

The architecture is designed as a set of distinct microservices that communicate through an **Apache Kafka event bus**. This event-driven approach decouples services, provides data replayability for robust testing, and scales to handle high-frequency data streams for professional traders. The system's standout feature is a sophisticated **Agentic AI Assistant**, which acts as the platform's "brain," enabling users to manage trading, research, and analysis via natural language commands. The core trading engine is **NautilusTrader**, a high-performance, Python-based platform with Rust components, designed for event-driven backtesting and live trading across all asset classes. The Agentic AI Assistant leverages **TradingAgent** for multi-agent decision-making, **OpenBB** for financial data integration, and **TA-Lib/ta-lib-python** (primary wrapper) & **Bukosabino/ta** (secondary wrapper) for technical analysis, forming a robust AI-driven trading brain.

The system incorporates advanced forecasting from **Stock-Prediction-Models** and **LSTM-Neural-Network-for-Time-Series-Prediction**, with real-time predictions enabled by **Real-time-stock-market-prediction** for live trading. **VectorBT** provides GPU-accelerated backtesting, while **TradingGym** complement NautilusTrader for flexible strategy development and simulated environments. Additional features include portfolio optimisation (**PyPortfolioOpt**, **Riskfolio-Lib**), anomaly detection (**PyOD**), no-code strategy building (**Blockly**), visualisation (**react-financial-charts**, **Plotly Dash**), explainable AI (**SHAP**), reinforcement learning (**FinRL**), and advanced NLP (**Transformers**, **PyTorch**), alongside options analytics (**QuantLib**).

Key Characteristics

- High Performance: Microsecond-level latency for trading operations
- Scalable: Horizontal scaling across multiple nodes
- Resilient: Fault-tolerant with automatic failover
- Secure: Enterprise-grade security with zero-trust architecture
- Observable: Comprehensive monitoring and alerting

Architecture Principles

1. Microservices Architecture

- Service Decomposition: Each business capability is a separate service
- Independent Deployment: Services can be deployed independently
- Technology Diversity: Services can use different technologies
- Fault Isolation: Failure in one service doesn't affect others

2. Event-Driven Architecture

- Asynchronous Communication: Services communicate via events
- Event Sourcing: All state changes are captured as events
- CQRS: Command Query Responsibility Segregation
- Event Streaming: Real-time event processing

3. Cloud-Native Design

- Container-First: All services are containerized
- Kubernetes Native: Designed for Kubernetes orchestration
- 12-Factor App: Follows 12-factor application principles
- Infrastructure as Code: All infrastructure is code-defined

4. API-First Design

- RESTful APIs: Standard REST interfaces
- GraphQL: Flexible query interface
- WebSocket: Real-time communication
- gRPC: High-performance inter-service communication

Core Services and Capabilities

1. Trading Engine

- System supports multiple asset classes (stocks, ETFs, futures, options, forex, and crypto).
- Purpose: Execute trading strategies and manage orders
- Technology: Python/Rust for performance-critical paths
- Key Features:
 - Order routing and execution
 - Strategy execution
 - Position management
 - Trade settlement

2. Portfolio Manager

- Purpose: Manage portfolios and asset allocation
- Technology: Python with NumPy/Pandas
- Key Features:
 - Portfolio optimization
 - Asset allocation
 - Performance attribution
 - Rebalancing

3. Risk Manager

- Purpose: Monitor and control trading risks
- Technology: Python with real-time processing
- Key Features:
 - Real-time risk monitoring
 - VaR calculations
 - Exposure limits
 - Stress testing
- Real-time Risk Dashboard: Visualises live risk metrics in the Next.js frontend.

4. Market Data Service

- Purpose: Collect, process, and distribute market data
- Technology: Python/Go for high throughput
- Key Features:
 - Real-time data feeds
 - Historical data storage
 - Data normalization
 - Market data distribution

5. Order Management System (OMS)

- Connects to various brokers, starting with Interactive Brokers, with basic order types (Market, Limit) as well as advanced and algorithmic order types (VWAP, TWAP).
- FIX Gateway for institutional connectivity.
- Purpose: Manage order lifecycle and execution
- Technology: Python with low-latency optimizations
- Key Features:
 - Order validation
 - Execution management
 - Fill processing
 - Compliance checks

6. Market Scanner Service

- **Purpose:** To provide real-time market scanning across a large universe of symbols based on user-defined criteria.
- **Technology:** Python/Go for high-throughput data processing.
- **Key Features:**
 - Consumes real-time data streams from Kafka.
 - Applies custom filters using technical indicators (e.g., TA-Lib).
 - Streams filtered results to a dedicated, high-performance UI grid.

7. AI-Powered Strategy Development: Users can develop strategies through multiple interfaces:

- **No-Code Strategy Builder:**
 - A visual drag-and-drop interface via Blockly.

- It is designed to generate clean, human-readable Python code, creating a powerful learning pathway for users to transition from visual design to programmatic customization.
 - **AI-Assisted Development & Debugging Agent:** Assists with Python code for strategies and integrations.
 - **Agentic AI Assistant:** Natural language commands for trading, research, and development assistance.
 - **Python Studio:** Traditional Python-based coding environment.
8. **Agentic AI Assistant:**
- Core user-interaction model via a Chatbot.
 - Uses an Agentic **Retrieval-Augmented Generation (RAG)** pipeline to process and query user-uploaded documents.
 - Composed of a collaborative network of specialised agents (Analyst, Researcher, Risk Manager, Compliance, etc.). **All inter-agent communication will occur asynchronously via the Apache Kafka event bus.** This ensures agents are fully decoupled, enables flexible, one-to-many information flows, and creates a complete, auditable log of the AI's reasoning process.
 - Users can ask the AI to run a backtest, get portfolio status, or even place live trades.
9. **Enterprise Security and Risk Management:** Platform is built with,
- **Zero-Trust architecture**
 - Real-time risk hub for pre-trade checks and account-level circuit breakers.
 - Role-based access control (**RBAC**)
 - Immutable audit trails for compliance in **Apache Iceberg**.
 - Feature Flags (e.g., **Unleash**) for dynamic toggling of strategies and kill-switches.
 - Static Application Security Testing (SAST) with **Bandit**, automatically scanning Python code for common security vulnerabilities.

The "Best-of-Breed" Integration Strategy

This strategy selects top open-source projects for each component, interconnected via an event-driven Kafka architecture, ensuring scalability, flexibility, and high performance for professional trading.

Core Technology Stack

The system is built on a modern, open-source technology stack. This integration strategy provides a strong foundation, but significant custom development is required to meet all enterprise-grade requirements.

- **Core Trading Engine: NautilusTrader**
 - **Role:** The undisputed choice for the institutional-grade engine handling backtesting, live trading, data management, and multi-asset support. It is a high-performance, production-grade algorithmic trading platform, providing quantitative traders with the ability to backtest portfolios of automated trading strategies on historical data with an event-driven engine, and also deploy those same strategies live, with no code changes. The platform is *AI-first*, designed to develop and deploy algorithmic trading strategies within a highly performant and robust Python-native environment. This helps to address the parity challenge of keeping the Python research/backtest environment consistent with the production live trading environment. It's design, architecture, and implementation philosophy prioritises software correctness and safety at the highest level, with the aim of supporting Python-native, mission-critical, trading system backtesting and live deployment workloads. The platform is also universal, and asset-class-agnostic — with any REST API or WebSocket feed able to be integrated via modular adapters. It supports high-frequency trading across a wide range of asset classes and instrument types including FX, Equities, Futures, Options, Crypto and Betting, enabling seamless operations across multiple venues simultaneously. The system integrates with TA-Lib/ta-lib-python (primary wrapper) & Bukosabino/ta (secondary wrapper) for indicators and OpenBB for data, enhancing the system's strategy validation capabilities.
 - **Rationale:** NautilusTrader provides superior performance for high-frequency trading, custom data integration via parquet, and an AI-first design, aligning with our Python-centric workflow. This establishes a powerful, two-stage research and validation pipeline: **VectorBT** will be used for broad, high-speed, vectorised research to identify promising strategies, while **NautilusTrader** will serve as the sole, high-fidelity, event-driven engine for realistic validation and live deployment. This approach eliminates architectural redundancy and solves the critical backtest-to-live parity challenge.
 - https://github.com/nautechsystems/nautilus_trader
 - https://github.com/nautechsystems/nautilus_ibapi

- **Data Pipeline:** While libraries like Unstructured.io and databases like **Qdrant** and **PostgreSQL** with **pgvector** will be used, the pipelines for data ingestion, resiliency, and fusion are custom logic that will be build.
 - **Apache Kafka:**
 - An open-source distributed event streaming platform used by thousands of companies for high-performance data pipelines, streaming analytics, data integration, and mission-critical applications.
 - Central event bus for real-time data, order flow, market ticks, AI decisions, and system events.
 - Decouples all services, provides data replayability for robust testing, and scales to handle high-frequency data streams for professional traders.
 - **Design Principle: Hierarchical Topic Architecture**
 - ✓ To ensure long-term scalability and simplify data governance, the system will implement a structured, hierarchical naming convention for all Kafka topics (e.g., domain.action.entity.source.symbol).
 - ✓ Consumer logic will be built to leverage wildcard subscriptions, allowing services to subscribe to broad or specific data streams efficiently (e.g., marketdata.tick.crypto.spot.binance.*), which simplifies service configuration and scales more effectively than a flat topic structure.
 - <https://github.com/apache/kafka>
 - **Schema Registry:**
 - Ensures data consistency across services.
 - It provides a serving layer for your metadata.
 - It stores a versioned history of all schemas based on a specified subject name strategy, provides multiple compatibility settings and allows evolution of schemas according to the configured compatibility settings and expanded support for these schema types.
 - It provides serialisers that plug into Apache Kafka clients that handle schema storage and retrieval for Kafka messages that are sent in any of the supported formats.
 - <https://github.com/confluentinc/schema-registry>
- **Infrastructure**
 - **Kubernetes:**
 - Kubernetes, also known as K8s, is an open source system for managing containerized applications across multiple hosts.
 - It provides basic mechanisms for the deployment, maintenance, and scaling of applications.
 - It builds upon a decade and a half of experience at Google running production workloads at scale using a system called Borg, combined with best-of-breed ideas and practices from the community.
 - <https://github.com/kubernetes/kubernetes>
 - **Docker:**
 - Docker is an open platform for developing, shipping, and running applications.
 - It enables you to separate your applications from your infrastructure so you can deliver software quickly.
 - With Docker, you can manage your infrastructure in the same ways you manage your applications.
 - By taking advantage of Docker's methodologies for shipping, testing, and deploying code, you can significantly reduce the delay between writing code and running it in production.
 - <https://github.com/docker-library/docker>
 - **Istio:**
 - Istio is an open-source service mesh that layers transparently onto existing distributed applications.
 - Istio's powerful features provide a uniform and more efficient way to secure, connect, and monitor services.
 - It is the path to load balancing, service-to-service authentication, and monitoring – with few or no service code changes.
 - It is an open platform for providing a uniform way to integrate microservices, manage traffic flow across microservices, enforce policies and aggregate telemetry data.

Istio's control plane provides an abstraction layer over the underlying cluster management platform, such as Kubernetes.

- Istio is composed of these components:
 - ✓ **Envoy:** Sidecar proxies per microservice to handle ingress/egress traffic between services in the cluster and from a service to external services. The proxies form a *secure microservice mesh* providing a rich set of functions like discovery, rich layer-7 routing, circuit breakers, policy enforcement and telemetry recording/reporting functions.
 - ✓ **Note:** The service mesh is not an overlay network. It simplifies and enhances how microservices in an application talk to each other over the network provided by the underlying platform.
 - ✓ **Istiod:** The Istio control plane. It provides service discovery, configuration and certificate management. It consists of the following sub-components:
 - ❖ **Pilot:** Responsible for configuring the proxies at runtime.
 - ❖ **Citadel:** Responsible for certificate issuance and rotation.
 - ❖ **Galley:** Responsible for validating, ingesting, aggregating, transforming and distributing config within Istio.
 - ✓ **Operator:** The component provides user friendly options to operate the Istio service mesh.
- <https://github.com/istio/istio>
- **NGINX:**
 - NGINX (pronounced "engine x" or "en-jin-eks") is the world's most popular Web Server, high performance Load Balancer, Reverse Proxy, API Gateway and Content Cache.
 - NGINX is comprised of individual modules, each extending core functionality by providing additional, configurable features.
 - NGINX modules can be built and distributed as static or dynamic modules.
 - Static modules are defined at build-time, compiled, and distributed in the resulting binaries.
 - <https://github.com/nginx/nginx>
- **Helm:**
 - Package management
 - Helm is a tool for managing Charts. Charts are packages of pre-configured Kubernetes resources.
 - Use Helm to:
 - ✓ Find and use popular software packaged as Helm Charts to run in Kubernetes
 - ✓ Share your own applications as Helm Charts
 - ✓ Create reproducible builds of your Kubernetes applications
 - ✓ Intelligently manage your Kubernetes manifest files
 - ✓ Manage releases of Helm packages
 - It is a tool that streamlines installing and managing Kubernetes applications. Think of it like apt/yum/homebrew for Kubernetes.
 - Helm renders your templates and communicates with the Kubernetes API.
 - It runs on your laptop, CI/CD, or wherever you want it to run.
 - Charts are Helm packages that contain at least two things:
 - ✓ A description of the package (Chart.yaml)
 - ✓ One or more templates, which contain Kubernetes manifest files
 - Charts can be stored on disk, or fetched from remote chart repositories (like Debian or RedHat packages).
 - <https://github.com/helm/helm>
- **Programming Languages**
 - **Python:** Primary language for business logic
 - **Rust:** Performance-critical components
 - **TypeScript:** Frontend and Node.js services
 - **Go:** Infrastructure and tooling
 - **SQL:** Database queries and procedures
- **Agentic AI Assistant: LangChain, LangGraph, TradingAgents, OpenBB, and TA-Lib/ta-lib-python & Bukosabino/ta**
 - **Role:** This combination will form the "brain" of the system.
 - **Rationale:**

- **LangChain** will serve as the underlying framework for building the Agentic Retrieval-Augmented Generation (Agentic RAG) capabilities, document processing, and the tool integration that powers the agents, as recommended in your features document.
 - ✓ <http://github.com/langchain-ai/langchain>
- **LangGraph** will manage the complex, asynchronous workflows between the AI Agents, Multi Agents, Agentic AI, data, and trading services and act as an Orchestrator.
 - ✓ <https://github.com/langchain-ai/langgraph>
- **TradingAgents** a multi-agent trading framework that mirrors the dynamics of real-world trading firms. By deploying specialised LLM-powered agents: from fundamental analysts, sentiment experts, and technical analysts, to trader, risk management team, the platform collaboratively evaluates market conditions and informs trading decisions. Moreover, these agents engage in dynamic discussions to pinpoint the optimal strategy. It provides a finance-specific foundation of Multi-Agents LLM Financial Trading Framework with specialised agent roles (analyst, researcher, risk manager) that directly map to the system requirements.
 - ✓ <https://github.com/TauricResearch/TradingAgents>
- **OpenBB** Platform offers access to equity, options, crypto, forex, macro economy, fixed income, and more while also offering a broad range of extensions to enhance the user experience according to their needs.
 - ✓ <http://github.com/OpenBB-finance/OpenBB>
- **TA-Lib/ta-lib-python** (primary wrapper) & **Bukosabino/ta** (secondary wrapper) provides a foundation for technical indicators, including custom volume-weighted ones.
 - ✓ <https://github.com/TA-Lib/ta-lib-python>
 - ✓ <https://github.com/bukosabino/ta>
- **OpenHands (formerly OpenDevin):** An AI-Assisted Development & Debugging Agent. It is a platform for software development agents powered by AI. It can do anything a human developer can: modify code, run commands, browse the web, call APIs, and even copy code snippets from StackOverflow. It is an AI agent specifically trained to assist with writing, debugging, and optimising Python code for trading strategies and system integrations, tailored to NautilusTrader, VectorBT, and custom APIs. It dramatically speeds up development, reduces bugs, and lowers the barrier to entry for quants who may be less familiar with the specific libraries being used. It transforms the AI from just a strategy generator into a full-fledged development partner.
 - ✓ **Features:**
 - (i) Speed up code reviews: Summarise pull requests, incorporate feedback, and push fixes so code reviews are quick and painless.
 - (ii) Refactor old code: Decompose monolithic code, refactor tech debt, and automate version bumps without breaking the build.
 - (iii) Expand test coverage: Generate tests for new features to elevate code quality and squash hidden bugs.
 - (iv) Fix failing pipelines: Fix broken tests without debugging or interrupting others for help.
 - (v) Create prototypes: Turn ideas and concepts into working code you can test with customers.
 - (vi) Get to "done" faster: Offload all the engineering toil you don't want to do to OpenHands.
 - (vii) Deeply customisable: OpenHands is an open-source core, unlocking customisation for a variety of enterprise use cases.
 - (viii) Best in class coding accuracy: OpenHands is a top performer across a variety of software development benchmarks for agents, like SWE-bench.
 - (ix) Works where you do: Use OpenHands in your browser, as a CLI, via API, or in tools like GitHub, GitLab, Slack, Jira, and more.
 - ✓ <https://github.com/All-Hands-AI/OpenHands>
- **Codename Goose:** an open source, extensible AI agent that goes beyond code suggestions - install, execute, edit, and test with any LLM.
 - ✓ **Features:**

- (i) Goose is your on-machine AI agent, capable of automating complex development tasks from start to finish. More than just code suggestions, goose can build entire projects from scratch, write and execute code, debug failures, orchestrate workflows, and interact with external APIs - autonomously.
- (ii) Whether you're prototyping an idea, refining existing code, or managing intricate engineering pipelines, goose adapts to your workflow and executes tasks with precision.
- (iii) Designed for maximum flexibility, goose works with any LLM and supports multi-model configuration to optimize performance and cost, seamlessly integrates with MCP servers, and is available as both a desktop app as well as CLI - making it the ultimate AI assistant for developers who want to move faster and focus on innovation.

✓ <https://github.com/block/goose>

▪ A Collaborative, Two-Agent Architecture

✓ The key to making this work is to assign specialized, non-overlapping roles to each agent. Based on their documented strengths, we can define a clear hierarchy:

✓ OpenHands: The "Specialist" Code-Level Agent

❖ OpenHands will be the master craftsman for **fine-grained, code-centric tasks**. Its entire domain is the Python code that powers your strategies and services. It excels at tasks requiring deep code understanding.

❖ Responsibilities:

1. Writing and debugging
2. NautilusTrader trading strategies.
3. Refactoring existing strategy code for optimization.
4. Expanding test coverage by writing pytest files.
5. Fixing bugs within a specific Python file or function.
6. Assisting with code that integrates with VectorBT, TA-Lib, and OpenBB.

✓ Codename Goose: The "General Contractor" Workflow Agent

❖ Goose will be the high-level orchestrator for **broad, multi-step, system-level engineering and DevOps workflows**. It excels at tasks that span multiple services, involve infrastructure, and require end-to-end autonomous execution.

❖ Responsibilities:

1. **DevOps and Infrastructure Management:** Setting up CI/CD pipelines, configuring Kafka topics, or managing Docker environments.
2. **Complex Task Automation:** Executing multi-step workflows like "run a backtest, if it succeeds, deploy the strategy to the paper trading environment, and notify me on Slack."
3. **Context-Aware Tool Routing:** Dynamically discovering and using the right tool for a job, which could include external APIs or even **invoking the OpenHands agent as one of its tools**.
4. **Data Migrations and Engineering:** Performing tasks like setting up a data pipeline to move historical data into ClickHouse or Apache Iceberg.

✓ Revised System Architecture Diagram

❖ The master orchestrator, LangGraph, sits at the center. It receives tasks and delegates them to the appropriate agent. Goose, in turn, can call OpenHands for specialized code modifications.

```

```mermaid
graph TD
 subgraph User Interface
 A[Lobe Chat UI]
 end
 end

```

```

subgraph AI Core
 B[LangGraph
Master Orchestrator]
 C[Codename Goose
Workflow & DevOps Agent]
 D[OpenHands
Specialist Code Agent]
end

subgraph Backend Services
 E[FastAPI Bridge]
 F[Apache Kafka Bus]
 G[Other Services
NautilusTrader, OpenBB, RAGFlow, etc.]
end

A --> |User Prompt| B
B --> |Routes DevOps/Workflow Tasks| C
B --> |Routes Code-Level Tasks| D
C --> |Can Invoke as a Tool| D
C --> |Executes Commands & API Calls| E
C --> |Interacts with| F
D --> |Modifies Code in Repositories| G
...

```

### ✓ **Revised Integration Strategy & Plan**

- ❖ To establish a collaborative AI core where LangGraph orchestrates tasks between Goose (for workflows) and OpenHands (for code), providing a unified experience through the Lobe Chat interface.

#### ❖ **Build Strategy**

##### 1. Deploy the Master Orchestrator (LangGraph):

- ✧ Set up the core LangGraph application within the /ai\_assistant service. This agent will be the single entry point for all AI tasks coming from Lobe Chat.
- ✧ Its primary job is intent recognition: to analyze the user's prompt and decide which specialized agent (Goose or OpenHands) is best suited for the task.

##### 2. Register Goose as a Tool:

- ✧ Integrate Codename Goose as a powerful "tool" that LangGraph can call.
- ✧ Goose's role will be exposed to LangGraph with a clear description: "Best for multi-step engineering workflows, DevOps, infrastructure tasks, and end-to-end automation."
- ✧ Configure Goose's MCP (Model Context Protocols) capabilities to connect to the FastAPI bridge, enabling it to interact with all backend services.
- ✧ Leverage Goose's ability to dynamically load tools via vector search. We will create a "tool library" of custom APIs (e.g., functions to interact with NautilusTrader's backtester) that Goose can discover and use autonomously.

##### 3. Register OpenHands as a Tool:

- ✧ Integrate OpenHands as another specialized tool available to LangGraph.
- ✧ Its role will be described as: "Best for writing, debugging, refactoring, and optimizing Python code, especially for trading strategies and data analysis."

#### ❖ **Leveraging Goose's Strengths**

This revised plan directly leverages the features you are interested in:

1. Task Automation & End-to-End Execution (1, 3, 7): This is Goose's primary role as the "General Contractor," handling complex workflows from start to finish.
  2. DevOps & Context-Aware Routing (2): Goose is designated for all DevOps tasks, and its ability to route to the correct tool (including other agents like OpenHands) is central to the architecture.
  3. MCP & Custom APIs (4, 6): Goose will use MCP to connect to the system's core APIs, and we will build a library of custom API tools specifically for it to use.
  4. Dynamic Tool Loading (5): This powerful feature means we can expand the system's capabilities just by adding new tools to a library, without having to reprogram the agent itself. Goose will find and use them as needed.
- ❖ By implementing this collaborative, two-agent architecture, you are not creating redundancy; you are building a more sophisticated, capable, and specialized AI workforce to power your trading system.
- **Predictions**
    - **Stock Prediction Models**
      - A collection of machine learning and deep learning models (e.g., ARIMA, LSTM, GANs) for stock forecasting, including trading bots and simulations.
      - These models improve forecasting accuracy for AI-driven trading strategies.
      - Works well with TradingAgent for decision-making and OpenBB for data integration.
      - <https://github.com/huseinzol05/Stock-Prediction-Models>
    - **LSTM – Neural Network for Time Series Prediction**
      - An LSTM model built with Keras for time series prediction, tested on sine waves and stock data.
      - LSTMs excel at time series forecasting, enhancing prediction capabilities.
      - Integrates seamlessly with TradingAgent's AI framework and TA-Lib/ta-lib-python (primary wrapper) & Bukosabino/ta's (secondary wrapper) data processing.
      - <https://github.com/jaungiers/LSTM-Neural-Network-for-Time-Series-Prediction>
    - **Real time stock market prediction**
      - A real-time prediction system using Tensorflow.js and Kafka for streaming data.
      - Relevant for live trading, adding real-time forecasting capabilities.
      - Complements NautilusTrader's live trading and OpenBB's data streams.
      - <https://github.com/victor369basu/Real-time-stock-market-prediction>
  - **Backtesting**
    - **VectorBT**
      - GPU-accelerated, vectorised backtesting.
      - It allows to easily backtest strategies with a couple of lines of Python code.
      - <https://github.com/polakowo/vectorbt>
    - **TradingGym**
      - It is a Python environment toolkit for training and backtesting the reinforcement learning (RL) agents for trading.
      - Enhances AI-driven strategy optimisation, aligning with the system's agentic RAFT goals.
      - Complements TradingAgent and FinRL for RL-based strategies.
      - <https://github.com/Yvictor/TradingGym>
  - **Options Analytics Engine: QuantLib**
    - **Role:** The go-to open-source library for all sophisticated options pricing, risk analytics, and calculations for complex derivatives.
    - **Rationale:** It is the premier library for quantitative finance, providing tools that are a powerful open-source alternative to proprietary systems. It will be integrated as a specialised service.
      - <https://github.com/lballabio/QuantLib>
      - <https://www.quantlib.org/>
  - **User Interface: Next.js**
    - **Role:** The unified "cockpit" that interacts with all backend services via a centralised API.

- **Rationale:** This is the modern standard for high-performance web applications and is specified in your features document. Use Next.js with react-financial-charts, Plotly Dash, ag-Grid, and a Real-time Risk Dashboard.
  - <https://github.com/vercel/next.js>
- **API & Orchestration Layer:**
  - **FastAPI:**
    - **Description:** It is a modern, fast (high-performance), web framework for building APIs with Python based on standard Python type hints.
    - **Features:**
      - ✓ It is a custom application that serves as the central bridge between the frontend, AI assistant, and backend trading engine.
      - ✓ Fast to code: Increase the speed to develop features by about 200% to 300%.
      - ✓ Fewer bugs: Reduce about 40% of human (developer) induced errors.
      - ✓ Intuitive: Great editor support. Completion everywhere. Less time debugging.
      - ✓ Easy: Designed to be easy to use and learn. Less time reading docs.
      - ✓ Short: Minimise code duplication. Multiple features from each parameter declaration. Fewer bugs.
      - ✓ Robust: Get production-ready code. With automatic interactive documentation.
    - <https://github.com/fastapi/fastapi>
  - **gRPC:**
    - Low latency streaming for market data.
    - High performance remote procedure call (RPC) framework that can run anywhere.
    - It enables client and server applications to communicate transparently, and simplifies the building of connected systems.
    - <https://github.com/grpc/grpc>
  - **LangGraph:**
    - Complex workflow orchestration.
- **Frameworks and Libraries**
  - **GraphQL:**
    - GraphQL is a query language for APIs and a runtime for fulfilling those queries with your existing data.
    - It provides a complete and understandable description of the data in your API, gives clients the power to ask for exactly what they need and nothing more, makes it easier to evolve APIs over time, and enables powerful developer tools.
    - <https://github.com/redwoodjs/graphql>
  - **React:**
    - It is a library for web and native user interfaces.
    - React is a JavaScript library for building user interfaces.
      - ✓ **Declarative:** React makes it painless to create interactive UIs. Design simple views for each state in your application and React will efficiently update and render just the right components when your data changes. Declarative views make your code more predictable, simpler to understand, and easier to debug.
      - ✓ **Component-Based:** Build encapsulated components that manage their own state, then compose them to make complex UIs. Since component logic is written in JavaScript instead of templates, you can easily pass rich data through your app and keep the state out of the DOM.
      - ✓ **Learn Once, Write Anywhere:** We don't make assumptions about the rest of your technology stack, so you can develop new features in React without rewriting existing code. React can also render on the server using Node and power mobile apps using React Native.
    - <https://github.com/facebook/react>
  - **Polars/Pandas/NumPy:** Data processing
  - **Asyncio:** Asynchronous programming
- **Chatbot Interface: Lobe Chat**
  - Modern design AI chat / LLMs UI framework.
  - Supports multiple AI providers (OpenAI / Claude 4 / Gemini / DeepSeek / Ollama / Qwen).
  - Knowledge Base (file upload / RAG ).
  - One click install MCP Marketplace and Artifacts / Thinking.

- Supports speech synthesis, multi-modal, and extensible (function call) plugin system.
- **Features:**
  - Smooth Conversation Experience: Fluid responses ensure a smooth conversation experience. It fully supports Markdown rendering, including code highlighting, LaTeX formulas, Mermaid flowcharts, and more.
  - Exquisite UI Design: With a carefully designed interface, it offers an elegant appearance and smooth interaction. It supports light and dark themes and is mobile-friendly.
  - Desktop App
  - Smart Online Search
  - Chain of Thought
  - Branching Conversations
  - File Upload / Knowledge Base
  - Multi-Model Service Provider Support
  - Local Large Language Model (LLM) Support
  - Model Visual Recognition
  - Text to Image Generation
  - Plugin System (Function Calling)
  - Progressive Web App (PWA) support provides a more native-like experience.
  - MCP Plugin One-Click Installation
  - MCP Marketplace
  - Artifacts Support
- <https://github.com/lobehub/lobe-chat>
- **Agentic RAG Pipeline: RAGFlow**
  - RAG (Retrieval-Augmented Generation) engine based on deep document understanding.
  - Streamlined RAG workflow for businesses of any scale, combining LLM (Large Language Models) to provide truthful question-answering capabilities, backed by well-founded citations from various complex formatted data.
  - **Features:**
    - Quality in, quality out:
      - ✓ Deep document understanding-based knowledge extraction from unstructured data with complicated formats.
      - ✓ Finds "needle in a data haystack" of literally unlimited tokens.
    - Template-based chunking:
      - ✓ Intelligent and explainable.
      - ✓ Plenty of template options to choose from.
    - Grounded citations with reduced hallucinations:
      - ✓ Visualisation of text chunking to allow human intervention.
      - ✓ Quick view of the key references and traceable citations to support grounded answers.
    - Compatibility with heterogeneous data sources:
      - ✓ Supports word, slides, excel, txt, images, scanned copies, structured data, web pages, and more.
    - Automated and effortless RAG workflow:
      - ✓ Streamlined RAG orchestration catered to both personal and large businesses.
      - ✓ Configurable LLMs as well as embedding models.
      - ✓ Multiple recall paired with fused re-ranking.
      - ✓ Intuitive APIs for seamless integration with business.
  - <https://github.com/infiniflow/ragflow>
- **Database:**
  - **PostgreSQL with pgvector:**
    - **Description:** The system uses **PostgreSQL** with the **pgvector** extension enabled, allowing it to function as an all-in-one database for both traditional relational data and vector embeddings for the AI.
    - **Features:**
      - ✓ Open-source vector similarity search for Postgres
      - ✓ Store your vectors with the rest of your data.
      - ✓ Supports:
        - (i) exact and approximate nearest neighbour search.

- (ii) single-precision, half-precision, binary, and sparse vectors.
  - (iii) L2 distance, inner product, cosine distance, L1 distance, Hamming distance, and Jaccard distance.
- <https://github.com/pgvector/pgvector>
- **ClickHouse:**
  - Column oriented database management system that allows generating analytical data reports in real-time.
  - High-speed, large-scale time-series analytics.
  - <https://github.com/ClickHouse/ClickHouse>
- **DuckDB:**
  - Fast, in-process OLAP queries for research.
  - High performance analytical database system.
  - Designed to be fast, reliable, portable, and easy to use.
  - Provides a rich SQL dialect, with support far beyond basic SQL.
  - Supports arbitrary and nested correlated subqueries, window functions, collations, complex types (arrays, structs, maps), and several extensions designed to make SQL easier to use.
  - <http://github.com/duckdb/duckdb>
- **Qdrant:**
  - **Description:** Qdrant is a high-performance, massive-scale Vector Database and Vector Search Engine.
  - **Features:**
    - ✓ It provides a production-ready service with a convenient API to store, search, and manage points – vectors with an additional payload.
    - ✓ It is tailored to extended filtering support.
    - ✓ It makes it useful for all sorts of neural-network or semantic-based matching, faceted search, and other applications.
    - ✓ Qdrant is written in Rust, which makes it fast and reliable even under high load.
    - ✓ With Qdrant, embeddings or neural network encoders can be turned into full-fledged applications for matching, searching, recommending, and much more.
  - <https://github.com/qdrant/qdrant>
- **Apache Iceberg:**
  - Long-term, immutable data storage.
  - High performance format for huge analytic tables.
  - Iceberg brings the reliability and simplicity of SQL tables to big data, while making it possible for engines like Spark, Trino, Flink, Presto, Hive, and Impala to safely work with the same tables, at the same time.
  - <https://github.com/apache/iceberg>
- **Redis:**
  - Caching and session storage.
  - For developers, who are building real-time data-driven applications, Redis is the preferred, fastest, and most feature-rich cache, data structure server, and document and vector query engine.
  - Redis excels in various applications, including:
    - ✓ Caching: Supports multiple eviction policies, key expiration, and hash-field expiration.
    - ✓ Distributed Session Store: Offers flexible session data modeling (string, JSON, hash).
    - ✓ Data Structure Server: Provides low-level data structures (strings, lists, sets, hashes, sorted sets, JSON, etc.) with high-level semantics (counters, queues, leaderboards, rate limiters) and supports transactions & scripting.
    - ✓ NoSQL Data Store: Key-value, document, and time series data storage.
    - ✓ Search and Query Engine: Indexing for hash/JSON documents, supporting vector search, full-text search, geospatial queries, ranking, and aggregations via Redis Query Engine.
    - ✓ Event Store & Message Broker: Implements queues (lists), priority queues (sorted sets), event deduplication (sets), streams, and pub/sub with probabilistic stream processing capabilities.

- ✓ Vector Store for GenAI: Integrates with AI applications (e.g. LangGraph, mem0) for short-term memory, long-term memory, LLM response caching (semantic caching), and retrieval augmented generation (RAG).
    - ✓ Real-Time Analytics: Powers personalization, recommendations, fraud detection, and risk assessment.
  - <https://github.com/redis/redis>
- **InfluxDB:**
  - It is a scalable datastore for metrics, events, and real-time analytics.
  - InfluxDB Core is a database built to collect, process, transform, and store event and time series data. It is ideal for use cases that require real-time ingest and fast query response times to build user interfaces, monitoring, and automation solutions.
  - Common use cases include:
    - ✓ Monitoring sensor data
    - ✓ Server monitoring
    - ✓ Application performance monitoring
    - ✓ Network monitoring
    - ✓ Financial market and trading analytics
    - ✓ Behavioral analytics
  - InfluxDB is optimised for scenarios where near real-time data monitoring is essential and queries need to return quickly to support user experiences such as dashboards and interactive user interfaces.
  - InfluxDB Core's feature highlights include:
    - ✓ Diskless architecture with object storage support (or local disk with no dependencies)
    - ✓ Fast query response times (under 10ms for last-value queries, or 30ms for distinct metadata)
    - ✓ Embedded Python VM for plugins and triggers
    - ✓ Parquet file persistence
    - ✓ Compatibility with InfluxDB 1.x and 2.x write APIs
    - ✓ Compatibility with InfluxDB 1.x query API (InfluxQL)
    - ✓ SQL query engine with support for FlightSQL and HTTP query API
  - <https://github.com/influxdata/influxdb>
- **MinIO/S3:**
  - MinIO is a high-performance, S3-compatible object storage solution released under the GNU AGPL v3.0 license.
  - It is designed for speed and scalability.
  - It powers AI/ML, analytics, and data-intensive workloads with industry-leading performance.
  - <https://github.com/minio/minio>
- **Order Management & Execution Layer:**
  - **FIX Gateway:**
    - Institutional-grade trading connectivity.
    - The Financial Information eXchange (FIX) protocol is a messaging standard developed specifically for the real-time electronic exchange of securities transactions.
    - FIX is a public-domain specification owned and maintained by FIX Protocol, Ltd (FPL).
    - **QuickFIX/J:**
      - ✓ Full featured messaging engine for the FIX protocol.
      - ✓ It is a 100% Java open source implementation of the popular C++ QuickFIX engine.
      - ✓ <https://github.com/quickfix-j/quickfixj>
    - **FIX8:**
      - ✓ A modern open-source C++ FIX framework featuring complete schema driven customisation, high performance and fast application development.
      - ✓ Comprised of a compiler for generating C++ message and field encoders, decoders and instantiation tables.
      - ✓ Runtime library to support the generated code and framework.
      - ✓ Set of complete client/server test applications.
      - ✓ <https://github.com/fix8/fix8>

- **Feature Store:**
  - **Feast:**
    - Open-source feature store for machine learning.
    - Fastest path to manage existing infrastructure to productionise analytic data for model training and online inference.
    - Allows ML platform teams to:
      - ✓ Make features consistently available for training and serving by managing an offline store (to process historical data for scale-out batch scoring or model training), a low-latency online store (to power real-time prediction), and a battle-tested feature server (to serve pre-computed features online).
      - ✓ Avoid data leakage by generating point-in-time correct feature sets so data scientists can focus on feature engineering rather than debugging error-prone dataset joining logic. This ensure that future feature values do not leak to models during training.
      - ✓ Decouple ML from data infrastructure by providing a single data access layer that abstracts feature storage from feature retrieval, ensuring models remain portable as you move from training models to serving models, from batch models to realtime models, and from one data infra system to another.
    - <https://github.com/feast-dev/feast>
  - **Tecton:**
    - Consistent feature management for training and inference.
    - <https://github.com/tecton-ai>
- **Observability and Monitoring Stack:**
  - **Prometheus:**
    - **Description:** It is a systems and service monitoring system. It collects metrics from configured targets at given intervals, evaluates rule expressions, displays the results, and can trigger alerts when specified conditions are observed.
    - **Features:**
      - ✓ A multi-dimensional data model (time series defined by metric name and set of key/value dimensions).
      - ✓ PromQL, a powerful and flexible query language to leverage this dimensionality.
      - ✓ No dependency on distributed storage; single server nodes are autonomous.
      - ✓ An HTTP pull model for time series collection.
      - ✓ Pushing time series is supported via an intermediary gateway for batch jobs.
      - ✓ Targets are discovered via service discovery or static configuration.
      - ✓ Multiple modes of graphing and dashboarding support.
      - ✓ Support for hierarchical and horizontal federation.
    - <https://github.com/prometheus/prometheus>
  - **Grafana:**
    - **Description:** It is an open and composable observability and data visualisation platform. Visualise metrics, logs, and traces from multiple sources like Prometheus, Loki, Elasticsearch, InfluxDB, Postgres, and many more. Grafana allows you to query, visualise, alert on and understand your metrics no matter where they are stored. Create, explore, and share dashboards with your team and foster a data-driven culture.
    - **Features:**
      - ✓ Visualisations: Fast and flexible client-side graphs with a multitude of options. Panel plugins offer many different ways to visualise metrics and logs.
      - ✓ Dynamic Dashboards: Create dynamic & reusable dashboards with template variables that appear as dropdowns at the top of the dashboard.
      - ✓ Explore Metrics: Explore your data through ad-hoc queries and dynamic drilldown. Split view and compare different time ranges, queries and data sources side by side.
      - ✓ Explore Logs: Experience the magic of switching from metrics to logs with preserved label filters. Quickly search through all your logs or streaming them live.
      - ✓ Alerting: Visually define alert rules for your most important metrics. Grafana will continuously evaluate and send notifications to systems like Slack, PagerDuty, VictorOps, OpsGenie.

- ✓ Mixed Data Sources: Mix different data sources in the same graph! You can specify a data source on a per-query basis. This works for even custom datasources.
  - <https://github.com/grafana/grafana>
- **Grafana Tempo:**
  - **Description:** High volume, minimal dependency distributed tracing for latency debugging. Distributed tracing helps teams quickly pinpoint performance issues and understand the flow of requests across services.
  - **Features:**
    - ✓ Cost-efficient, requiring only object storage to operate.
    - ✓ Deeply integrated with Prometheus, Grafana, and Loki.
    - ✓ The Traces Drilldown UI simplifies this process by offering a user-friendly interface to view and analyse trace data, making it easier to identify and resolve issues without needing to write complex queries.
  - <https://github.com/grafana/tempo>
- **Memray:**
  - **Description:** A memory profiler for Python from Bloomberg. It can track memory allocations in Python code, in native extension modules, and in the Python interpreter itself. It detects leaks in Python code. It can generate several different types of reports to help analyse the captured memory usage data. While commonly used as a CLI tool, it can also be used as a library to perform more fine-grained profiling tasks.
  - **Justification:** In a high-performance trading system, memory leaks or inefficient memory usage can degrade performance over time and lead to system instability. Memray is invaluable for debugging these issues, especially in long-running services like the trading engine and AI assistant.
  - **Features:**
    - ✓ Traces every function call so it can accurately represent the call stack, unlike sampling profilers.
    - ✓ Also handles native calls in C/C++ libraries so the entire call stack is present in the results.
    - ✓ Blazing fast. Profiling slows the application only slightly. Tracking native code is somewhat slower, but this can be enabled or disabled on demand.
    - ✓ It can generate various reports about the collected memory usage data, like flame graphs.
    - ✓ Works with Python threads.
    - ✓ Works with native-threads (e.g. C++ threads in C extensions).
    - ✓ It can analyse allocations in applications to help discover the cause of high memory usage.
    - ✓ It can find memory leaks and hotspots in code that cause a lot of allocations.
  - <https://github.com/bloomberg/memray>
- **Elasticsearch:**
  - It is a distributed search and analytics engine, scalable data store and vector database optimised for speed and relevance on production-scale workloads.
  - Elasticsearch is the foundation of Elastic's open Stack platform.
  - Search in near real-time over massive datasets, perform vector searches, integrate with generative AI applications, and much more.
  - Use cases enabled by Elasticsearch include:
    - ✓ Retrieval Augmented Generation (RAG)
    - ✓ Vector search
    - ✓ Full-text search
    - ✓ Logs
    - ✓ Metrics
    - ✓ Application performance monitoring (APM)
    - ✓ Security logs
    - ✓ ... and more!
  - <https://github.com/elastic/elasticsearch>
- **Grafana Loki:**

- Loki is a horizontally scalable, highly-available, multi-tenant log aggregation system inspired by Prometheus.
- It is designed to be very cost effective and easy to operate.
- It does not index the contents of the logs, but rather a set of labels for each log stream.
- Compared to other log aggregation systems, Loki:
  - ✓ does not do full text indexing on logs. By storing compressed, unstructured logs and only indexing metadata, Loki is simpler to operate and cheaper to run.
  - ✓ indexes and groups log streams using the same labels you're already using with Prometheus, enabling you to seamlessly switch between metrics and logs using the same labels that you're already using with Prometheus.
  - ✓ is an especially good fit for storing Kubernetes Pod logs. Metadata such as Pod labels is automatically scraped and indexed.
  - ✓ has native support in Grafana (needs Grafana v6.0).
- A Loki-based logging stack consists of 3 components:
  - ✓ Alloy is agent, responsible for gathering logs and sending them to Loki.
  - ✓ Loki is the main service, responsible for storing logs and processing queries.
  - ✓ Grafana for querying and displaying the logs.
- Note that Alloy replaced Promtail in the stack, because Promtail is considered to be feature complete, and future development for logs collection will be in Grafana Alloy.
- Loki is like Prometheus, but for logs
  - ✓ A multidimensional label-based approach to indexing.
  - ✓ A single-binary, easy to operate system with no dependencies.
- Loki differs from Prometheus by focusing on logs instead of metrics, and delivering logs via push, instead of pull.
- <https://github.com/grafana/loki>
- **Jaeger:**
  - It is a Distributed Tracing System.
  - Distributed tracing observability platforms, such as Jaeger, are essential for modern software applications that are architected as microservices.
  - Jaeger maps the flow of requests and data as they traverse a distributed system.
  - These requests may make calls to multiple services, which may introduce their own delays or errors.
  - Jaeger connects the dots between these disparate components, helping to identify performance bottlenecks, troubleshoot errors, and improve overall application reliability.
  - It utilises OpenTelemetry Collector framework as the base and extends it to implement Jaeger's unique features.
  - It brings significant improvements and changes, making Jaeger more flexible, extensible, and better aligned with the OpenTelemetry project.
  - OpenTelemetry is the de-facto standard for application instrumentation providing the foundation for observability.
  - Jaeger is now based on the cornerstone of this project, the OpenTelemetry Collector.
  - It is a complete tracing platform that includes storage and the UI.
  - OpenTelemetry Collector is usually an intermediate component in the collection pipelines that are used to receive, process, transform, and export different telemetry types.
  - <https://github.com/jaegertracing/jaeger>
- **AlertManager:**
  - The Alertmanager handles alerts sent by client applications such as the Prometheus server.
  - It takes care of deduplicating, grouping, and routing them to the correct receiver integrations such as email, PagerDuty, OpsGenie, or many other mechanisms thanks to the webhook receiver.
  - It also takes care of silencing and inhibition of alerts.
  - <https://github.com/prometheus/alertmanager>
- **ELK Stack:**
  - It is an Elastic stack (ELK) powered by Docker and Compose.
  - It gives you the ability to analyse any data set by using the searching/aggregation capabilities of Elasticsearch and the visualisation power of Kibana.
  - <https://github.com/deviantony/docker-elk>
- **Enterprise Security and Risk Management:**

- **Authentication & Authorization**
  - OAuth 2.0/OpenID Connect: Standard authentication
  - JWT Tokens: Stateless authentication
  - RBAC: Role-based access control
  - API Keys: Service-to-service authentication
- **Data Protection**
  - Encryption at Rest: AES-256 encryption
  - Encryption in Transit: TLS 1.3
  - Key Management: HashiCorp Vault
  - Secret Management: Kubernetes secrets
- **Network Security**
  - Network Policies: Kubernetes network policies
  - Service Mesh: Istio for service-to-service security
  - Ingress Security: NGINX with security headers
  - DDoS Protection: Rate limiting and throttling
- **Zero-Trust** architecture.
- Real-time risk hub.
- Immutable audit trails in Apache Iceberg.
- **Unleash:**
  - **Description:** Feature Flags for dynamic toggling of strategies and kill-switches.
  - **Features:**
    - ✓ Powerful open-source solution for feature management.
    - ✓ Streamlines development workflow.
    - ✓ Accelerates software delivery.
    - ✓ Empowers teams to control how and when they roll out new features to end users.
    - ✓ Deploys code to production in smaller, more manageable releases at user's own pace.
    - ✓ Feature flags let users test their code with real production data, reducing the risk of negatively impacting the users' experience.
    - ✓ Enables the users' team to work on multiple features simultaneously without the need for separate feature branches.
  - <https://github.com/Unleash/unleash>
- **Static Application Security Testing (SAST) with Bandit:**
  - **Description:** Add a SAST tool like **Bandit** into the development lifecycle. This tool automatically scans Python code for common security vulnerabilities.
  - **Justification:** This shifts security "left", finding and fixing potential vulnerabilities *before* they ever reach production. It's a critical component of a modern DevSecOps pipeline and essential for a system handling financial transactions.
  - **Features:**
    - ✓ Tool designed to find common security issues in Python code.
    - ✓ Processes each file, builds an AST from it, and runs appropriate plugins against the AST nodes.
    - ✓ After scanning all the files, it generates a report.
  - <https://github.com/PyCQA/bandit>
- **Enhanced Data Visualisation:**
  - **React-financial-charts for advanced financial charting**
    - **Description:** A React library for advanced financial charting, supporting candlestick, OHLC, and volume charts.
    - **Features:**
      - ✓ Customisable charts for stocks, options, and other assets.
      - ✓ Interactive features like zooming and tooltips.
      - ✓ Integrates with Next.js for dynamic UI updates.
    - **Benefit:** Enhances the Next.js frontend with professional-grade financial visualisations, improving user experience for traders analysing market data.
    - <https://github.com/react-financial/react-financial-charts>
  - **Plotly Dash for interactive dashboards**
    - **Description:** A Python framework for building interactive web-based dashboards, compatible with Next.js via API integration.

- **Features:**
      - ✓ Interactive visualisations for market data, portfolio performance, and risk metrics.
      - ✓ Supports real-time updates and user-driven data exploration.
      - ✓ Integrates with pandas for seamless data handling.
    - **Benefit:** Adds dynamic dashboards to complement TradingView charts, enhancing data exploration for traders and analysts.
    - <https://github.com/plotly/dash>
  - **"Glass Box" UI / Decision Event Explorer**
    - **Description:** A dedicated UI component that provides a "glass box" view into the system's operations by visualizing the flow of Kafka events associated with a user's actions.
    - **Features:**
      - ✓ Trace the complete, end-to-end causal chain for every trade, from the initial AI insight to the final broker confirmation.
      - ✓ Provide a verifiable audit trail for compliance and a powerful debugging tool for strategy development.
    - **Benefit:** Transforms the system's complexity into a transparent, auditable, and trustworthy strength, building profound user confidence.
- **Hyperparameter Tuning:**
  - **Optuna:**
    - **Description:** Best-in-class hyperparameter tuning framework to optimise strategies alongside the backtesting engine. It is an automatic hyperparameter optimisation software framework, designed for machine learning. It features an imperative, *define-by-run* style user API, due to which the code written with Optuna enjoys high modularity, and the user of Optuna can dynamically construct the search spaces for the hyperparameters.
    - **Features:**
      - ✓ Lightweight, versatile, and platform agnostic architecture: Handle a wide variety of tasks with a simple installation that has few requirements.
      - ✓ Pythonic search spaces: Define search spaces using familiar Python syntax including conditionals and loops.
      - ✓ Efficient optimisation algorithms: Adopt state-of-the-art algorithms for sampling hyperparameters and efficiently pruning unpromising trials.
      - ✓ Easy parallelisation: Scale studies to tens or hundreds of workers with little or no changes to the code.
      - ✓ Quick visualisation: Inspect optimisation histories from a variety of plotting functions.
    - <https://github.com/optuna/optuna>
- **Portfolio Optimisation:**
  - **PyPortfolioOpt**
    - **Description:** A Python library for portfolio optimisation, offering tools for asset allocation and risk management.
    - **Features:**
      - ✓ Supports mean-variance optimisation, Black-Litterman allocation, and risk parity.
      - ✓ Calculates risk metrics like volatility and Sharpe ratio.
      - ✓ Integrates with pandas for data input from OpenBB or NautilusTrader.
    - **Benefit:** Enhances risk management by optimising portfolio allocations, complementing the existing risk checks.
    - <https://github.com/robertmartin8/PyPortfolioOpt>
  - **Riskfolio-Lib for asset allocation and risk analysis**
    - **Description:** A Python library for advanced portfolio optimisation and risk analysis.
    - **Features:**
      - ✓ Supports hierarchical risk parity, CVaR optimisation, and worst-case scenario analysis.
      - ✓ Provides visualisation tools for risk contributions and portfolio weights.
      - ✓ Integrates with pandas for data compatibility.
    - **Benefit:** Adds sophisticated risk analysis tools, enhancing the system's enterprise-grade risk management capabilities.
    - <https://github.com/dcajasn/Riskfolio-Lib>
- **Anomaly Detection:**

- **PyOD for detecting anomalies in trading data and system performance**
    - **Description:** A Python library for outlier detection, suitable for identifying anomalies in trading data or system performance.
    - **Features:**
      - ✓ Supports multiple anomaly detection algorithms (e.g., Isolation Forest, AutoEncoder).
      - ✓ Integrates with pandas and NumPy for data processing.
      - ✓ Provides visualisation tools for anomaly analysis.
    - **Benefit:** Enhances system reliability by detecting unusual trading patterns or performance issues, complementing Prometheus and Grafana.
    - <https://github.com/yzhao062/pyod>
- **Explainable AI:**
  - **SHAP for transparent AI-driven trading decisions**
    - **Description:** A Python library for explainable AI, providing tools to interpret machine learning model predictions.
    - **Features:**
      - ✓ Generates SHAP values to explain model outputs (e.g., trading decisions).
      - ✓ Supports visualisation of feature importance and decision impacts.
      - ✓ Integrates with machine learning frameworks like scikit-learn and TensorFlow.
    - **Benefit:** Improves transparency of AI-driven trading decisions, aligning with regulatory compliance and user trust.
    - <https://github.com/shap/shap>
- **No-Code Strategy Builder:**
  - **Blockly for visual programming of trading strategies**
    - **Description:** A JavaScript library for visual programming, enabling drag-and-drop interfaces for building applications.
    - **Features:**
      - ✓ Supports custom block definitions for trading strategies (e.g., indicators, conditions, actions).
      - ✓ Integrates with Next.js for web-based no-code interfaces.
      - ✓ Generates executable code from visual blocks.
    - **Benefit:** Enhances the no-code strategy builder, making the system accessible to non-programmers.
    - <https://github.com/google/blockly>
- **Python Studio:** Traditional coding environment.
- **Advanced NLP for AI Assistant:**
  - **Hugging Face's Transformers for advanced financial text analysis**
    - **Description:** A Python library for state-of-the-art NLP models, including pre-trained transformers for financial applications.
    - **Features:**
      - ✓ Supports fine-tuning models like FinBERT for financial text analysis (e.g., news, earnings calls).
      - ✓ Integrates with LangChain for enhanced AI assistant capabilities.
      - ✓ Provides tools for sentiment analysis and text generation.
    - **Benefit:** Enhances the AI assistant's ability to process financial texts, improving decision-making and user interaction.
    - <https://github.com/huggingface/transformers>
- **DeepLearning:**
  - **PyTorch:**
    - **Description:** A deep learning framework. Tensors and Dynamic neural networks in Python with strong GPU acceleration.
    - **Features:**
      - ✓ Supports custom AI model development for trading strategies.
      - ✓ Offers flexibility for building and training models tailored for the system.
    - **Benefit:** It provides two high-level features,
      - ✓ Tensor computation (like NumPy) with strong GPU acceleration.
      - ✓ Deep neural networks built on a tape-based autograd system.
    - <https://github.com/pytorch/pytorch>

- **Reinforcement Learning:**
  - **FinRL for optimising trading strategies**
    - **Description:** A Python framework for reinforcement learning in financial trading, offering pre-built models and environments.
    - **Features:**
      - ✓ Supports deep reinforcement learning for trading strategy optimisation.
      - ✓ Integrates with OpenBB and TA-Lib/ta-lib-python (primary wrapper) & Bukosabino/ta (secondary wrapper) for data and indicator inputs.
      - ✓ Provides environments for backtesting and live trading.
    - **Benefit:** Enhances AI-driven strategy development, complementing TradingAgent's multi-agent framework.
    - <https://github.com/AI4Finance-Foundation/FinRL>
- **Custom Development/Integration:**
  - **Advanced Features:** Most advanced features require custom logic, including the AI-powered opportunity detection, post-trade analytics, enterprise security guardrails, advanced risk management hub, and the complete user interface.
- **Managing Updates to Cloned Repositories**

To balance the benefits of upstream updates with the stability of your customised system, adopt a **selective update strategy** for cloned repositories (e.g., OpenHands, Codename Goose, LangChain, LangGraph, kafka, etc.).

  - **Maintain Forked Repositories:**
    - Treat cloned repositories as forks, customized for your system (e.g., integrating OpenHands with Kafka).
    - Document customizations in a changelog to track differences.
    - Focus on improving your fork for domain-specific needs (e.g., SDLC workflows, compliance for Lawyer Agent).
  - **Monitor Upstream Repositories:**
    - Use GitHub Actions to check for updates daily, notifying via Slack if changes are detected.
    - Example: Monitor OpenHands for security patches or new features.
  - **Selective Update / Integration Process:**
    - Prioritize updates for security patches, bug fixes, or relevant features.
    - Test updates in a staging environment using a CI/CD pipeline (e.g., GitHub Actions, Jenkins).
    - Require human review for major updates to assess alignment with your system.
  - **Dependency Management:**
    - Use Dependabot to monitor and update dependencies within cloned repositories.
    - Pin dependencies in lock files (e.g., requirements.txt) to prevent conflicts.
    - Run automated tests post-update to ensure compatibility.
  - **Comprehensive Testing and Validation:**
    - Implement a CI/CD pipeline with pytest, Jest, and Cypress to test updates.
    - Use LangSmith to evaluate agent performance after updates.
  - **Rollback Capability:**
    - Capability to roll back the upgrade / integration in case of any failed updates.
  - **Contribute Back (Optional):**
    - Submit generalizable customizations to upstream repositories to reduce future merge conflicts.
  - **Benefits:**
    - Preserves your customizations while allowing critical updates.
    - Automates monitoring and testing, reducing manual effort.
    - Ensures dependency compatibility and system stability.
  - **Notification Frequency:**
    - Daily: Silent logging only (no notifications).
    - Weekly: Single consolidated message with ALL updates.
    - Monthly: Integration status and deployment summary.
    - Emergency: Immediate notification and Integration for critical security issues only.

## Architecture

Refer “/docs/comprehensive\_system\_architecture.md” file for the comprehensive details of System Architecture.

### **Open-Source Repositories:**

1. [https://github.com/nautechsystems/nautilus\\_trader](https://github.com/nautechsystems/nautilus_trader)
2. [https://github.com/nautechsystems/nautilus\\_ibapi](https://github.com/nautechsystems/nautilus_ibapi)
3. <https://github.com/apache/kafka>
4. <https://github.com/confluentinc/schema-registry>
5. <https://github.com/kubernetes/kubernetes>
6. <https://github.com/docker-library/docker>
7. <https://github.com/istio/istio>
8. <https://github.com/nginx/nginx>
9. <https://github.com/helm/helm>
10. <http://github.com/langchain-ai/langchain>
11. <https://github.com/langchain-ai/langgraph>
12. <https://github.com/TauricResearch/TradingAgents>
13. <http://github.com/OpenBB-finance/OpenBB>
14. <https://github.com/TA-Lib/ta-lib-python>
15. <https://github.com/bukosabino/ta>
16. <https://github.com/All-Hands-AI/OpenHands>
17. <https://github.com/block/goose>
18. <https://github.com/huseinzol05/Stock-Prediction-Models>
19. <https://github.com/jaungiers/LSTM-Neural-Network-for-Time-Series-Prediction>
20. <https://github.com/victor369basu/Real-time-stock-market-prediction>
21. <https://github.com/polakowo/vectorbt>
22. <https://github.com/Yvictor/TradingGym>
23. <https://github.com/lballabio/QuantLib>
24. <https://github.com/vercel/next.js>
25. <https://github.com/fastapi/fastapi>
26. <https://github.com/grpc/grpc>
27. <https://github.com/redwoodjs/graphql>
28. <https://github.com/facebook/react>
29. <https://github.com/lobehub/lobe-chat>
30. <https://github.com/infiniflow/ragflow>
31. <https://github.com/pgvector/pgvector>
32. <https://github.com/ClickHouse/ClickHouse>
33. <http://github.com/duckdb/duckdb>
34. <https://github.com/qdrant/qdrant>
35. <https://github.com/apache/iceberg>
36. <https://github.com/redis/redis>
37. <https://github.com/influxdata/influxdb>
38. <https://github.com/minio/minio>
39. <https://github.com/quickfix-j/quickfixj>
40. <https://github.com/fix8/fix8>
41. <https://github.com/feast-dev/feast>
42. <https://github.com/tecton-ai>
43. <https://github.com/prometheus/prometheus>
44. <https://github.com/grafana/grafana>
45. <https://github.com/grafana/tempo>
46. <https://github.com/bloomberg/memray>
47. <https://github.com/elastic/elasticsearch>
48. <https://github.com/grafana/loki>
49. <https://github.com/jaegertracing/jaeger>
50. <https://github.com/prometheus/alertmanager>
51. <https://github.com/deviantony/docker-elk>
52. <https://github.com/Unleash/unleash>
53. <https://github.com/PyCQA/bandit>
54. <https://github.com/react-financial/react-financial-charts>
55. <https://github.com/plotly/dash>
56. <https://github.com/optuna/optuna>

57. <https://github.com/robertmartin8/PyPortfolioOpt>
58. <https://github.com/dcajahn/Riskfolio-Lib>
59. <https://github.com/yzhao062/pyod>
60. <https://github.com/shap/shap>
61. <https://github.com/google/blockly>
62. <https://github.com/huggingface/transformers>
63. <https://github.com/pytorch/pytorch>
64. <https://github.com/AI4Finance-Foundation/FinRL>

### **Detailed Six Phase Prompt and Integration Plan**

This plan outlines the objectives and key outcomes for each of the six primary development phases, building upon the foundational work established in Phase 0 (Dependency Management). This integration plan outlines a structured, phased approach to developing a robust, enterprise-grade Algorithmic Trading System (ATS). It leverages the existing infrastructure (Docker, Kubernetes, monitoring stack) and partially implemented components (NautilusTrader, AI assistant, security) while addressing gaps and introducing advanced features. Each phase includes detailed objectives, durations, tasks, integration focus, and deliverables to ensure clarity and completeness.

#### ***Phase 0: Dependency Management Setup Prompt:***

You are an expert full-stack software engineering agent tasked with building a world-class, enterprise-grade Algorithmic Trading System (ATS). The current system is approximately 20-25% complete, featuring a robust infrastructure (Docker, Kubernetes, monitoring stack) and partially implemented components (trading engine, AI assistant, security), as outlined in the "Current Status - Comprehensive Analysis" document.

#### **Core Directive: Incremental & Context-Aware Development**

For every task, you MUST follow this protocol:

1. **Analyse:** Review the existing codebase to understand its current state and implementation details.
2. **Compare:** Assess the current implementation against the requirements, designs, and tasks specified in this prompt and subsequent phase-specific prompts.
3. **Execute:**
  - If the feature is fully implemented and aligned with specifications, report its completion and proceed.
  - If the feature is partially implemented or misaligned, modify the existing code to meet the new requirements.
  - If the feature is absent, develop it from scratch, adhering to the system's architectural principles.

#### **Architectural Principles**

- **Microservices Architecture:** Design all components as independently deployable services.
- **Event-Driven Architecture:** Use Apache Kafka as the central event bus for asynchronous communication between services.
- **Cloud-Native Design:** Ensure all services are containerized with Docker and orchestrated via Kubernetes.
- **API-First Design:** Expose functionality through well-defined APIs, including REST (with versioning), GraphQL, WebSocket, and gRPC.

#### **Dependency Management**

- Dependencies must be containerized; avoid global installations.
- Each service maintains its own dependency configuration (e.g., `requirements.txt` for Python, `package.json` for Node.js) and a corresponding `Dockerfile`.

#### **Documentation Context**

All subsequent prompts will provide you with three sets of instructions derived directly from the official project documentation:

- **Requirements:** Specify the "what" and "why" for each feature or component.
- **Designs:** Detail the architectural "how" for implementation.
- **Tasks:** Provide granular, step-by-step checklists for execution.

#### **Six-Phase Integration Plan**

1. **Phase 0:** Dependency Management Setup - Establish management for 60+ components with automated monitoring and testing.
2. **Phase 1:** Core System Validation & Hardening - Stabilize security, trading engine, and APIs.
3. **Phase 2:** Frontend and Broker Integration - Develop UI and integrate Interactive Brokers and Alpaca.

4. **Phase 3: AI/ML Integration** - Implement AI-driven features and advanced analytics.
5. **Phase 4: Frontend & Live Trading** - Enable live trading and real-time UI integration.
6. **Phase 5: Enterprise Readiness** - Add observability, security, and compliance features.
7. **Phase 6: System Enhancement & Future-Ready Technologies** - Introduce advanced capabilities and optimize for production.

### Instructions

- Start with Phase 0 and progress sequentially through Phase 6.
- Use the existing system as the foundation, enhancing or modifying only where features are missing or misaligned.
- Ensure all developments align with the microservices, event-driven, and cloud-native architecture.
- Refer to attached documents ("/docs/complete\_requirements.md", "/docs/complete\_designs.md", "/docs/complete\_tasks.md") for the respective phase's detailed guidance.
- Ensure to use Docker for all dependencies. Do not install any dependencies and libraries globally.

### Integration Plan - Phase 0: Dependency Management Setup

- **Objective:**
  - Establish a comprehensive dependency management framework for over 60+ best-of-breed components **and elevate it to a continuous, core operational process.** This ensures long-term stability, security, and scalability, mitigating integration risks throughout the system's lifecycle.
- **Key Tasks:**
  - **Repository Management:** Fork and organize 50+ repositories (e.g., NautilusTrader, LangChain, OpenHands) into tiers: Tier 1 (Critical), Tier 2 (Important), Tier 3 (Supporting), and Tier 4 (Infrastructure), following the structure in "/docs/comprehensive\_dependency\_workflow.md".
  - **Automated Monitoring:** Configure GitHub Actions for tiered monitoring—daily checks for Tier 1-2, weekly for Tier 3-4—with silent logging and immediate security alerts for vulnerabilities.
  - **Notification System:** Implement multi-channel notifications (Teams, Discord, Email) with weekly summary reports and real-time emergency alerts for critical updates or security issues.
  - **Testing Environment:** Develop Docker-based testing environments integrated with the existing CI/CD pipeline to validate dependency updates automatically.
  - **Automated Integration Testing:** Implement a robust, automated integration testing suite within the existing CI/CD pipeline. This suite must validate updates to critical components (e.g., NautilusTrader, LangGraph) in an isolated environment before they are approved for merging to ensure system-wide stability.
  - **Customization Tracking:** Maintain detailed changelogs for customizations in each fork (e.g., Kafka enhancements in OpenHands) using a dedicated customization management system.
  - **Health Dashboard:** Create a real-time dependency health dashboard integrated with the existing monitoring stack (Prometheus, Grafana) to track repository status, update impacts, and manual controls.
- **Integration Focus:**
  - Utilize the current containerized setup (Docker) and orchestration (Kubernetes) for seamless integration from "/docs/dependency\_management\_guide.md".
  - Adopt the selective update strategy from "/Other Files/Dependency Management - SDLC Multi Agent System.docx", prioritizing security patches and feature enhancements relevant to the ATS.
- **Deliverables:**
  - A fully operational dependency management system with automated monitoring, testing, and notification capabilities.
  - A dependency health dashboard seamlessly integrated into the existing observability framework.

**Acknowledge:** Confirm that you have understood this directive and are ready to begin development.

### *Phase 1: Immediate Priority - Core System Validation & Hardening Prompt:*

#### Objective

- Stabilize the existing system by validating security measures, testing the trading engine and databases, and enhancing the API layer to address critical gaps.

#### Key Tasks

- **Security Validation:** Resolve dependency conflicts, implement zero-trust architecture validation, and integrate fraud detection with behavioral analytics.
- **Trading Engine Testing:** Validate NautilusTrader with comprehensive tests for order management, risk controls, and multi-asset support.
- **API Enhancement:** Enhance FastAPI with REST (versioned), GraphQL, gRPC, and WebSocket endpoints for real-time data.
- **Database Integration:** Ensure reliable connections to PostgreSQL (pgvector), ClickHouse, DuckDB, and Qdrant with performance and failover testing.
- **Testing Framework:** Establish a Docker-based testing framework for all components, integrated with CI/CD.
- **Performance Testing:** Conduct load and stress tests for sub-millisecond latency and scalability.
- **Integration Testing:** Validate end-to-end workflows, data consistency, and error recovery.
- **Documentation:** Generate detailed system documentation and implement audit trails for compliance.

### Integration Instructions

- Build upon the existing NautilusTrader engine, Kafka setup, and security features (zero-trust, fraud detection).
- Enhance or develop new API endpoints based on current implementations.

### Expected Outcome

- A hardened core system with a validated trading engine, APIs, and databases, fully tested and documented.

### References

- ["/docs/complete\\_requirements.md"](#) (Phase 1 requirements)
- ["/docs/complete\\_designs.md"](#) (Phase 1 design)
- ["/docs/complete\\_tasks.md"](#) (Phase 1 tasks)

### Integration Plan - Phase 1: Immediate Priority - Core System Validation & Hardening

- **Objective:**
  - Strengthen the existing system by validating security, testing core trading and database components, and enhancing the API layer to address critical gaps identified in the "Current Status - Comprehensive Analysis" document.
  - Establish the core infrastructure, including the trading engine, data pipelines, and initial security measures, building on the existing 20-25% implementation.
  - To stabilise the existing foundation by resolving security conflicts, conducting comprehensive testing of the core trading and database components, and implementing an enhanced, multi-protocol API layer.
  - This phase addresses the critical gaps identified in the current status analysis, focusing on making the existing components robust and interconnected.
- **Key Tasks:**
  - **Security Validation:** Enforce a zero-trust architecture, resolve dependency conflicts, and integrate fraud detection with behavioral analytics for robust protection.
  - **Trading Engine Testing:** Conduct comprehensive testing of NautilusTrader, validating order management, risk controls, and multi-asset support (e.g., equities, forex, crypto).
  - **API Enhancement:** Develop a multi-protocol API layer using FastAPI, supporting REST (with versioning), GraphQL, gRPC, and WebSocket for real-time data streaming.
  - **Database Integration:** Establish reliable, performant connections to PostgreSQL (with pgvector), ClickHouse, DuckDB, and Qdrant, including failover and performance testing.
  - **Testing Framework:** Build a Docker-based framework for isolated component testing, integrated with the existing CI/CD pipeline.
  - **Performance Testing:** Perform load and stress tests to achieve sub-millisecond latency and validate horizontal scalability.
  - **Integration Testing:** Ensure end-to-end workflow consistency, data integrity, and error recovery across all core components.
  - **Documentation & Compliance:** Produce detailed system documentation and implement an audit trail system for regulatory compliance.

- **Kafka Architecture Enhancement:** Design and implement the hierarchical topic naming convention for the Apache Kafka event bus. Refactor initial service configurations to align with this new, scalable architecture.
- **Integration Focus:**
  - Enhance the existing NautilusTrader engine, Kafka event bus, and security features (zero-trust, fraud detection) rather than rebuilding from scratch.
  - Upgrade or develop new API endpoints as needed, building on any existing implementations.
- **Deliverables:**
  - A stabilized core system with validated trading engine, data pipelines, and security measures.
  - Comprehensive test suites and documentation ensuring operational reliability and compliance.

**Acknowledge:** Confirm that you have understood this directive and are ready to begin Phase 1 development.

### ***Phase 2: Frontend and Broker Integration Prompt:***

#### **Objective:**

- Develop multi-platform user interfaces and integrate Interactive Brokers and Alpaca to enable trading functionality, addressing missing frontend and trading capabilities.

#### **Key Tasks:**

- **Frontend Development:** Build a responsive UI with React 18 and TypeScript, including trading dashboard, order management, and portfolio views.
- **Broker Integration:** Implement Interactive Brokers (TWS/Gateway) and Alpaca integrations with order routing, market data feeds, and a unified abstraction layer.
- **Advanced UI:** Integrate Lobe Chat (AI assistant) and Blockly (no-code builder) into the frontend.
- **Real-Time Streaming:** Set up a WebSocket system for real-time market data distribution.
- **Desktop, Mobile & PWA:** Develop a PWA, React Native mobile app, and an Electron Desktop app with push notifications and biometric authentication.
- **Testing:** Create comprehensive test suites for frontend, broker integrations, and UI components.

#### **Integration Instructions:**

- Connect the frontend to the existing API layer and Kafka bus from Phase 1.
- Enhance any existing broker-related code with new integrations.

#### **Expected Outcome:**

- A fully functional frontend across web, mobile, and desktop with live trading and real-time data capabilities.

#### **References:**

- `"/docs/complete_requirements.md"` (Phase 2 requirements)
- `"/docs/complete_designs.md"` (Phase 2 design)
- `"/docs/complete_tasks.md"` (Phase 2 tasks)

### **Integration Plan - Phase 2: Frontend and Broker Integration**

- **Objective:**
  - To build the complete user-facing interfaces across web, mobile, and desktop platforms, and to implement the critical broker integration layer for Interactive Brokers and Alpaca.
  - This phase will address the "Frontend Completely Missing" and "No Actual Trading Capability" findings from the status audit.
- **Key Tasks:**
  - **Frontend Development:** Build a modern, responsive UI using React 18 and TypeScript, featuring a trading dashboard, order management, and portfolio visualization.
  - **Broker Integration:** Implement Interactive Brokers (TWS/Gateway) and Alpaca integrations, including order routing, market data feeds, and a unified broker abstraction layer for intelligent routing.
  - **Advanced UI Features:** Incorporate Lobe Chat for an AI assistant and Blockly for a no-code strategy builder within the frontend.
  - **No-Code to Clean Code Pipeline:** Ensure the Blockly-based strategy builder generates well-commented, human-readable Python code that aligns with the system's core strategy classes. This

- creates a seamless "graduation" path for users moving from the visual interface to the Python Studio.
  - **Real-Time Streaming:** Develop a WebSocket-based system for efficient, real-time market data distribution to the UI.
  - **Desktop, Mobile & PWA:** Create a Progressive Web App (PWA), a React Native mobile app and an Electron desktop app with push notifications and biometric authentication.
  - **Testing:** Develop comprehensive test suites for frontend components, broker integrations, and UI responsiveness.
- **Integration Focus:**
  - Connect the frontend to the enhanced API layer (FastAPI) and Kafka bus established in Phase 1.
  - Leverage any existing broker-related code, extending it with new integrations as required.
- **Deliverables:**
  - A fully functional, multi-platform frontend with live trading capabilities and real-time data visualization.
  - A unified broker abstraction layer supporting multiple brokerage services.

**Acknowledge:** Confirm that you have understood this directive and are ready to begin Phase 2 development.

### *Phase 3: AI/ML Integration Prompt:*

#### **Objective:**

- Integrate advanced AI and Machine Learning capabilities, including agentic frameworks, predictive models, and real-time analytics, to create an intelligent trading system.

#### **Key Tasks:**

- **LangChain/LangGraph:** Implement document processing and multi-agent workflows for trading research.
- **TradingAgents:** Deploy specialized agents (Analyst, Risk Manager, Trader) with role-based decision-making.
- **Inference Engine:** Set up a real-time inference engine (e.g., TensorFlow Serving, PyTorch) for sub-millisecond predictions.
- **Advanced AI/ML:** Integrate FinRL (reinforcement learning), PyOD (anomaly detection), Optuna (hyperparameter tuning), and Transformers (NLP).
- **Sentiment Analysis:** Build a pipeline for news, social media, and financial document sentiment analysis.
- **Testing:** Validate AI components, model performance, and integration with trading workflows.

#### **Integration Instructions:**

- **Enhance the Proactive RAG Intelligence Engine:**
  - Elevate the existing RAG pipeline in ``/ai_assistant`` from a reactive tool into a proactive intelligence engine.
  - Create a dedicated "Researcher" agent tasked with continuously monitoring external data sources (e.g., news feeds, SEC filing APIs), automatically processing new documents via RAGFlow, and updating the vector database (Qdrant or pgvector) in real-time.
  - This ensures all other agents are always accessing the most current, pre-processed knowledge base.
- **Connect AI agents** to the trading engine and other services via the **Kafka event bus** and the API layer.

#### **Expected Outcome:**

- An AI-driven trading system with real-time analytics and validated decision-making capabilities.

#### **References:**

- `"/docs/complete_requirements.md"` (Phase 3 requirements)
- `"/docs/complete_designs.md"` (Phase 3 design)
- `"/docs/complete_tasks.md"` (Phase 3 tasks)

### **Integration Plan - Phase 3: AI/ML Integration**

- **Objective:**
  - To integrate the full suite of AI and Machine Learning capabilities, transforming the platform into an intelligent trading system.

- This involves moving beyond the currently implemented functional AI assistant to include predictive models and advanced agentic frameworks.
- **Key Tasks:**
  - **LangChain/LangGraph:** Implement document processing, multi-agent workflows, and coordination for trading research and strategy development.
  - **TradingAgents:** Deploy specialized agents (Analyst, Risk Manager, Trader) with role-based decision-making capabilities.
  - **Event-Driven Agent Communication:** Refactor the Agentic AI Assistant's architecture to ensure all inter-agent communication is managed asynchronously through the Kafka event bus, defining a clear and extensible event schema for agent interactions.
  - **Inference Engine:** Establish a high-performance, real-time model inference engine (e.g., TensorFlow Serving, PyTorch) targeting sub-millisecond latency.
  - **Advanced AI/ML:** Integrate FinRL (reinforcement learning), PyOD (anomaly detection), Optuna (hyperparameter tuning), and Transformers (NLP) for enhanced analytics.
  - **Sentiment Analysis:** Develop a pipeline for analyzing news, social media, and financial documents to gauge market sentiment.
  - **Testing & Validation:** Validate AI components, model accuracy, and integration with trading workflows through rigorous testing.
- **Integration Focus:**
  - Build upon the existing AI assistant and RAG pipeline in the /ai\_assistant directory.
  - Link AI agents to the trading engine and API layer via Kafka for seamless data flow.
- **Deliverables:**
  - An AI-driven trading system with real-time analytics, multi-agent coordination, and validated predictive models.

**Acknowledge:** Confirm that you have understood this directive and are ready to begin Phase 3 development.

#### *Phase 4: Frontend & Live Trading Prompt:*

##### **Objective:**

- Integrate the frontend with a live trading backend for real-time order execution, position management, and analytics via the UI.

##### **Key Tasks:**

- **Live Trading:** Implement WebSocket and REST APIs for broker connectivity and order execution.
- **Trading Interface:** Enable order placement, modification, and cancellation through the frontend.
- **Data Visualization:** Integrate TradingView with real-time updates and technical indicators.
- **Risk Management:** Develop a real-time risk monitoring and control interface.
- **Analytics Dashboard:** Implement performance metrics calculation and visualization.
- **Security:** Secure the frontend with MFA and RBAC.
- **Testing:** Conduct end-to-end testing of live trading workflows and UI integration.

##### **Integration Instructions:**

- Wire the frontend to the trading engine, API layer, and broker integrations.
- Enhance existing security measures with MFA and RBAC.

##### **Expected Outcome:**

- A seamless live trading experience with real-time data and analytics fully integrated into the frontend.

##### **References:**

- "/docs/complete\_requirements.md" (Phase 4 requirements)
- "/docs/complete\_designs.md" (Phase 4 design)
- "/docs/complete\_tasks.md" (Phase 4 tasks)

#### **Integration Plan - Phase 4: Frontend & Live Trading**

- **Objective:**
  - To connect the fully featured frontend from Phase 2 to a live trading backend, enabling real-time order execution, position management, and performance analytics through the UI.

- **Key Tasks:**
  - **Live Trading:** Implement WebSocket and REST APIs for real-time broker connectivity and order execution.
  - **Trading Interface:** Enable order placement, modification, and cancellation directly from the frontend.
  - **Data Visualization:** Integrate TradingView charting with real-time updates and customizable technical indicators.
  - **Risk Management:** Provide a real-time risk monitoring and control interface within the dashboard.
  - **Analytics Dashboard:** Develop a performance analytics dashboard with metrics calculation and visualization.
  - **"Glass Box" UI Implementation:** Develop and integrate the "Decision Event Explorer" into the frontend, allowing users to visualize the complete Kafka event chain for their trading activities, enhancing transparency and trust.
  - **Security:** Secure the frontend with multi-factor authentication (MFA) and role-based access control (RBAC).
  - **Testing:** Perform end-to-end testing of live trading workflows and frontend-backend synchronization.
- **Integration Focus:**
  - Connect frontend components to the trading engine, API layer, and broker integrations from previous phases.
  - Enhance any existing authentication mechanisms with MFA and RBAC.
- **Deliverables:**
  - A seamless, real-time trading experience with live data visualization and analytics integrated into the frontend.
  - Fully validated live trading workflows across all platforms.

**Acknowledge:** Confirm that you have understood this directive and are ready to begin Phase 4 development.

#### ***Phase 5: Enterprise Readiness Prompt:***

##### **Objective:**

- Elevate the platform to enterprise-grade status with monitoring, security, compliance, and scalability features for institutional use.

##### **Key Tasks:**

- **Observability:** Deploy Prometheus, Grafana, Jaeger, and ELK Stack for comprehensive monitoring.
- **Security:** Implement OAuth2/OIDC, SSO, RBAC, and Apache Iceberg for immutable audit trails.
- **High Availability:** Set up multi-region deployment with automated disaster recovery.
- **Performance:** Optimize with intelligent auto-scaling and resource management.
- **Enterprise Integration:** Integrate LDAP/Active Directory, Feast/Tecton feature stores, and Unleash for feature flags.
- **Compliance:** Automate regulatory reporting and enforce data retention policies.
- **Testing:** Validate with security audits, performance tests, and compliance checks.

##### **Integration Instructions:**

- Enhance existing security (zero-trust, fraud detection) with enterprise-grade controls.
- Integrate observability tools with the trading engine, API, and AI components.

##### **Expected Outcome:**

- A production-ready system with enterprise-grade monitoring, security, and compliance features.

##### **References:**

- ["/docs/complete\\_requirements.md"](#) (Phase 5 requirements)
- ["/docs/complete\\_designs.md"](#) (Phase 5 design)
- ["/docs/complete\\_tasks.md"](#) (Phase 5 tasks)

#### **Integration Plan - Phase 5: Enterprise Readiness**

- **Objective:**
  - Elevate the platform to enterprise-grade status with comprehensive monitoring, advanced security, compliance features, and scalability for institutional deployment.
- **Key Tasks:**
  - **Observability:** Deploy Prometheus, Grafana, Jaeger, and ELK Stack for metrics, tracing, and logging across all components.
  - **Security:** Implement advanced authentication (OAuth2/OIDC, SSO), RBAC, and immutable audit trails using Apache Iceberg.
  - **High Availability:** Configure multi-region deployment with automated disaster recovery and failover mechanisms.
  - **Performance:** Optimize with intelligent auto-scaling and resource management for peak efficiency.
  - **Enterprise Integration:** Integrate with LDAP/Active Directory, feature stores (Feast/Tecton), and Unleash for feature flag management.
  - **Compliance:** Automate regulatory reporting and enforce data retention policies for audit readiness.
  - **Testing:** Conduct security audits, performance testing, and compliance validation to ensure enterprise standards.
  - **Market Scanner Implementation:** Develop and integrate a real-time Market Scanner Service and corresponding UI, providing a powerful, customizable tool for opportunity discovery, similar to TradeStation's RadarScreen.
- **Integration Focus:**
  - Enhance existing security features (zero-trust, fraud detection) with advanced enterprise-grade controls.
  - Integrate observability tools with the trading engine, API layer, and AI components from prior phases.
- **Deliverables:**
  - A production-ready system with robust monitoring, security, and compliance capabilities.
  - Validated enterprise features ensuring scalability and institutional reliability.

**Acknowledge:** Confirm that you have understood this directive and are ready to begin Phase 5 development.

#### *Phase 6: System Enhancement & Future-Ready Technologies Prompt:*

##### **Objective:**

- Enhance the platform with next-generation features, including ultra-low latency, advanced AI, market analysis, extended broker support, and future-ready interfaces.

##### **Key Tasks:**

- **Infrastructure:** Implement a high-performance message bus, advanced caching, and low-latency networking.
- **Order Management:** Develop a smart order router and execution algorithms (TWAP, VWAP, Iceberg).
- **Market Analysis:** Build order book analytics, liquidity detection, and market impact estimation tools.
- **Broker Expansion:** Integrate OANDA, Coinbase, and FIX protocol (QuickFIX/J, FIX8).
- **Kubernetes Deployment:** Implement full deployment with Helm charts, Istio, and GitOps CI/CD.
- **Future Features:** Add voice trading, AR interfaces, Memray profiling, and Grafana Tempo tracing.
- **Validation:** Conduct end-to-end testing, chaos engineering, and production readiness certification.

##### **Integration Instructions:**

- Extend the broker layer with OANDA, Coinbase, and FIX support.
- Optimize for sub-millisecond latency using Phase 5 enhancements.

##### **Expected Outcome:**

- A future-ready trading platform with advanced features and institutional-grade connectivity.

##### **References:**

- ["/docs/complete\\_requirements.md"](#) (Phase 6 requirements)
- ["/docs/complete\\_designs.md"](#) (Phase 6 design)

- `"/docs/complete_tasks.md"` (Phase 6 tasks)

## Integration Plan - Phase 6: System Enhancement & Future-Ready Technologies

- **Objective:**
  - Enhance the platform with cutting-edge features, including ultra-low latency infrastructure, advanced AI, market microstructure analysis, extended broker support, and next-generation interfaces.
- **Key Tasks:**
  - **Infrastructure:** Implement a high-performance message bus, advanced caching, and low-latency networking for optimal speed.
  - **Order Management:** Develop a smart order router and execution algorithms (e.g., TWAP, VWAP, Iceberg) for sophisticated trading.
  - **Market Analysis:** Build real-time market microstructure analysis tools, including order book analytics, liquidity detection, and market impact estimation.
  - **Broker Expansion:** Integrate OANDA, Coinbase, and FIX protocol (QuickFIX/J, FIX8) for institutional connectivity.
  - **Kubernetes Deployment:** Fully deploy with Kubernetes using Helm charts, Istio service mesh, and GitOps CI/CD workflows.
  - **Future Features:** Add voice trading and AR interfaces, integrate Memray profiling and Grafana Tempo tracing, **develop a Customer Service Bot using the RAG pipeline for automated user support, and scope the development of a Strategy Marketplace with copy trading capabilities for a future release.**
  - **Validation:** Perform end-to-end testing, chaos engineering, and production readiness certification.
- **Integration Focus:**
  - Extend the broker layer from Phase 2 with OANDA, Coinbase, and FIX protocol support.
  - Optimize all components for sub-millisecond latency, building on Phase 5's performance enhancements.
- **Deliverables:**
  - A future-ready trading platform with advanced features, institutional-grade connectivity, and certified production readiness.

**Acknowledge:** Confirm that you have understood this directive and are ready to begin Phase 6 development.